
SPAR: Stress-Penalized Adaptive Routing for Balanced Sparse MoE

Aviral Bhardwaj* Ananya Jain* Ansh Agrawal* Mann Thakkar*

Department of Computer Science
University of Toronto
Toronto, Ontario, Canada

{aviral.bhardwaj, ananya.jain, ansh.agrawal, mann.thakkar}@mail.utoronto.ca

Abstract

Sparse Mixture-of-Experts (MoE) models scale capacity by activating only a small subset of experts per token, significantly reducing per-token compute. However, in the **LoRA-MoE** setting, this efficiency is undermined by an initialization bottleneck: adapters typically begin as nearly identical mappings, so a standard router receives little early task signal to distinguish experts. This often leads to rapid routing collapse, even with auxiliary balancing losses. While standard MoE approaches use auxiliary objectives to enforce load balance, these methods are sensitive to hyperparameter tuning and can suppress the specialization they are meant to encourage in **LoRA-MoE**. We present **SPAR** (Stress-Penalized Adaptive Routing), an auxiliary-loss-free method that applies a symmetric, zero-equilibrium load correction directly to router logits outside the gradient graph. By decoupling load control from the language-modeling objective, SPAR stabilizes training without interfering with task gradients. We evaluate SPAR on a frozen Qwen2-1.5B backbone against standard top- k routing, DeepSeek-style bias balancing, and Expert Choice routing. SPAR achieves a leading perplexity in Wikitext-103 and Pile datasets while permitting structured, non-uniform expert utilization (effective experts ≈ 7.2 - 7.9), offering a SOTA alternative to traditional auxiliary-loss balancing in LoRA-based MoE training.

1 Introduction and Problem Statement

Sparse Mixture-of-Experts (MoE) architectures scale model capacity without proportionally increasing per-token compute by routing each token to a small subset of k out of N experts [1, 2]. However, this efficiency depends critically on the router. Without explicit load control, training exhibits *routing collapse*, a feedback loop in which dominant experts accumulate stronger updates and further reinforce their dominance, leaving remaining experts undertrained [2, 3]. This design underpins modern large-scale systems such as Mixtral [4], which achieves strong performance while activating only a sparse subset of experts per token.

Standard approaches, most notably the Switch Transformer [2] and GShard [5], augment the training objective with an auxiliary loss that penalizes uneven expert utilization. The balancing coefficient must be tuned manually, and no single value works reliably across different training stages or model scales. Auxiliary-loss-free alternatives such as DeepSeek’s Loss-Free Balancing [6, 7] update expert biases online to steer load, but remain vulnerable to collapse when the router gate specializes faster than the bias correction can respond. This tension is compounded when experts are LoRA adapters

*Equal contribution. Code: <https://github.com/UofT-CSC490-W2026/T-MoE>

[8]. MixLoRA [9] observes that LoRA-based MoE models exhibit greater routing instability than full-parameter counterparts, a consequence of standard zero- B initialization, which renders all experts functionally identical at the start of training and deprives the router of any signal to differentiate between them.

We propose **SPAR** (Stress-Penalized Adaptive Routing), a method that enforces load balance directly in the routing scores rather than through a loss term. SPAR applies a symmetric load correction outside the gradient graph where overloaded experts are penalized in proportion to their excess load, and underloaded experts receive a proportional boost, with the correction exactly zero at equilibrium. Load control and task optimization are fully decoupled, with a single free hyperparameter controlling routing sharpness and no auxiliary loss coefficient to tune.

Key Contributions.

- **Elimination of Auxiliary-Loss Fragility:** SPAR removes the need for hyperparameter tuning by enforcing load balance directly within the routing logits.
- **Decoupled Gradient Dynamics:** Through an out-of-graph, zero-equilibrium correction, SPAR decouples load control from task learning to prevent objective conflict.
- **State-of-the-Art Performance:** SPAR achieves a leading perplexity while permitting structured expert specialization.

2 Related Work

A dominant line of work addresses routing collapse by augmenting the training objective with a load-balancing auxiliary loss. Switch Transformer and GShard introduced variants of this approach based on assignment frequencies and routing probabilities [2, 5], and it has since become the default in most MoE systems. However, the balancing coefficient must be tuned for each setting: too small and collapse proceeds regardless; too large and routing becomes artificially uniform, suppressing the specialization that makes MoE valuable in the first place. This sensitivity is the core limitation that motivates auxiliary-loss-free approaches.

Recent methods of auxiliary-loss-free approaches decouple load control from the gradient graph entirely. Expert Choice re-frames the routing problem so that each expert selects its own top tokens under a fixed capacity budget, guaranteeing balance by construction [10]. DeepSeekMoE explores architectural refinements such as fine-grained expert segmentation and shared experts to improve utilization [11], while DeepSeek’s Loss-Free Balancing maintains a per-expert bias updated online based on observed utilization, steering load without any loss term [6, 7]. A primal-dual interpretation of this update family has also been studied [12]. LPR [13] improves load uniformity at inference time via variational prototype routing, with modest quality trade-offs. While these methods avoid gradient competition, they introduce their own limitations: Expert Choice can drop tokens when capacity is exhausted, and bias-based correction can lag behind fast-moving router dynamics during warmup.

Regardless, most of these methods require either full-model fine-tuning or conversion from a dense model to MoE, both of which are computationally expensive. In contrast, parameter-efficient fine-tuning has motivated a parallel line of work that combines sparse routing with LoRA adapters [8]. MixLoRA, MoLoRA, LoRAMoE, and related methods inject multiple LoRA experts into frozen transformer layers and use a shared router to specialize them across tasks [9, 14, 15]. Compared with full-parameter MoE, this regime consistently exhibits greater routing instability: the frozen backbone already provides strong shared representations, while the adapter updates are initially near-zero, giving the router little signal to differentiate between experts early in training. Existing methods in this space rely on auxiliary losses inherited from full-parameter MoE without addressing this initialization-specific instability. We address this gap with SPAR, described in detail in Section 4.

3 Preliminaries

In this section, we introduce the foundational components of Mixture-of-Experts (MoE) in Section 3.1, Low-Rank Adaptation (LoRA) in Section 3.2, and describe the LoRA-MoE architecture in Section 3.3.

3.1 Mixture-of-Experts (MoE).

A Mixture-of-Experts layer replaces a single feed-forward network with N parallel expert networks $\{f_i\}_{i=1}^N$, each processing a subset of tokens. A router assigns each token x to a subset of experts, and the layer output is a weighted combination of their outputs:

$$\text{MoE}(x) = \sum_{i=1}^N w_i \cdot f_i(x), \quad (1)$$

where $w_i \geq 0$ and $\sum_i w_i = 1$. In practice, sparse top- k routing activates only $k \ll N$ experts per token, keeping compute proportional to k rather than N . The central challenge in MoE training is load balancing: without explicit control, a small subset of experts tends to dominate, leaving others undertrained.

3.2 Low-Rank Adaptation (LoRA).

Fine-tuning large pretrained models is computationally expensive when all parameters are updated. LoRA [8] addresses this by freezing the pretrained weights and injecting trainable low-rank perturbations. For a frozen weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, the adapted output is

$$h = W_0 x + \frac{\alpha}{r} B A x, \quad (2)$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are the trainable factors, $r \ll \min(d_{\text{in}}, d_{\text{out}})$ is the rank, and α/r is a fixed scaling factor. B is zero-initialized so the adapter contributes no delta at the start of training. Only A and B are updated; W_0 remains frozen throughout.

3.3 LoRA-MoE Architecture

We construct a sparse MoE layer by replacing selected feed-forward sublayers of a frozen pretrained transformer with a LoRA-MoE layer. Each expert f_i is a LoRA-adapted copy of the original MLP: the frozen pretrained weights are shared across experts as non-trainable buffers, and each expert maintains its own independent low-rank factors (A_i, B_i) . The output of the layer for a token x is

$$\text{LoRA-MoE}(x) = \sum_{i \in \mathcal{S}(x)} w_i \left(W_0 x + \frac{\alpha}{r} B_i A_i x \right), \quad (3)$$

where $\mathcal{S}(x) \subseteq [N]$ is the set of k experts selected by the router for token x , and w_i are the corresponding mixture weights. The frozen base $W_0 x$ is shared across all experts; only the low-rank deltas $B_i A_i$ are expert-specific and trainable.

This design has two practical advantages. First, the parameter overhead scales as $O(N \cdot r \cdot (d_{\text{in}} + d_{\text{out}}))$, which is small relative to the frozen backbone. Second, because all experts start from the same zero-delta initialization, expert symmetry is preserved at the start of training, giving the router a fair signal before any expert has differentiated.

MoE layers are inserted at every other transformer block, replacing the feed-forward sublayer at those positions. The backbone weights at all other positions remain frozen and unchanged.

4 Model Architecture

Most parameter-efficient adaptation methods apply a single shared adapter to all inputs, treating the pretrained feed-forward network as a monolithic transformation. We instead distribute adaptation capacity across N specialized experts, each a low-rank perturbation of the same frozen base network. A learned router maps each input token to a sparse subset of k experts, and the layer output is a weighted combination of their adapted representations. The frozen backbone is never modified; all trainable parameters reside in the per-expert low-rank factors and the router prototypes. The router is **SPAR** (Stress-Penalized Adaptive Routing), which controls load balance directly in the routing scores without any auxiliary loss (Figure 1).

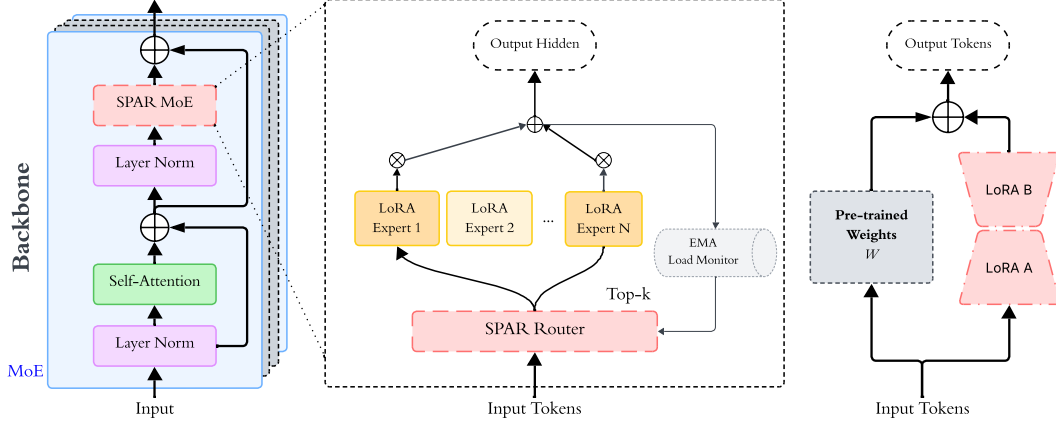


Figure 1: LoRA MoE with SPAR architecture. (a) Transformer block with SPAR MoE replacing the frozen MLP at selected layers. (b) MoE dispatch: the SPAR Router assigns tokens to top- k LoRA experts with EMA-based load correction. (c) SwiGLU LoRA expert: a low-rank adapter delta is added to the frozen pre-trained weights.

4.1 SPAR Routing

Routing Formulation. Let $x \in \mathbb{R}^D$ be a token activation and $W_i \in \mathbb{R}^D$ the unit-normalized prototype vector of expert i . SPAR computes load-corrected routing logits as

$$z_i(x) = \cos(x, W_i) - \lambda \left(L_i - \frac{k}{N} \right), \quad (4)$$

where $L_i \in [0, k]$ is a smoothed estimate of expert i 's dispatch fraction and $\lambda > 0$ is a fixed penalty scale. The correction is symmetric: experts dispatched above their fair share k/N are penalized; underloaded experts receive an equal boost. Since $\sum_{i=1}^N (L_i - k/N) = 0$ identically, the correction redistributes routing signal without changing the total logit budget. The top- k experts under these corrected logits are selected (Figure 2).

When load is perfectly balanced ($L_i = k/N$ for all i), the correction vanishes and routing reduces to pure cosine similarity. The penalty is only active when needed, unlike an auxiliary loss which applies gradient pressure regardless of the current load state.

Mixture weights are computed separately from the selection step using uncorrected cosine similarities over the selected set:

$$w_i = \frac{\exp(\cos(x, W_i)/\tau)}{\sum_{j \in \mathcal{S}(x)} \exp(\cos(x, W_j)/\tau)}, \quad i \in \mathcal{S}(x), \quad (5)$$

where $\tau > 0$ is a temperature parameter and $\mathcal{S}(x)$ is the selected expert set. Load correction governs *which* experts are selected; mixture weights reflect only representational alignment. Including the load penalty in the weights would suppress the contribution of overloaded experts even for tokens they serve well, distorting the aggregated output.

Load Tracking via EMA. Expert load is tracked as an exponential moving average of hard dispatch fractions. At each optimizer step, the instantaneous load fraction of expert i is

$$U_i = \frac{1}{B \cdot S} \sum_{\text{tokens}} \mathbf{1}[\text{token selects expert } i], \quad (6)$$

where B is batch size and S is sequence length. Each token selects k experts, so $\sum_{i=1}^N U_i = k$ identically and perfect balance corresponds to $U_i = k/N$, consistent with the fair share in Equation 4. The EMA update is

$$L_i \leftarrow (1 - \alpha) L_i + \alpha U_i, \quad (7)$$

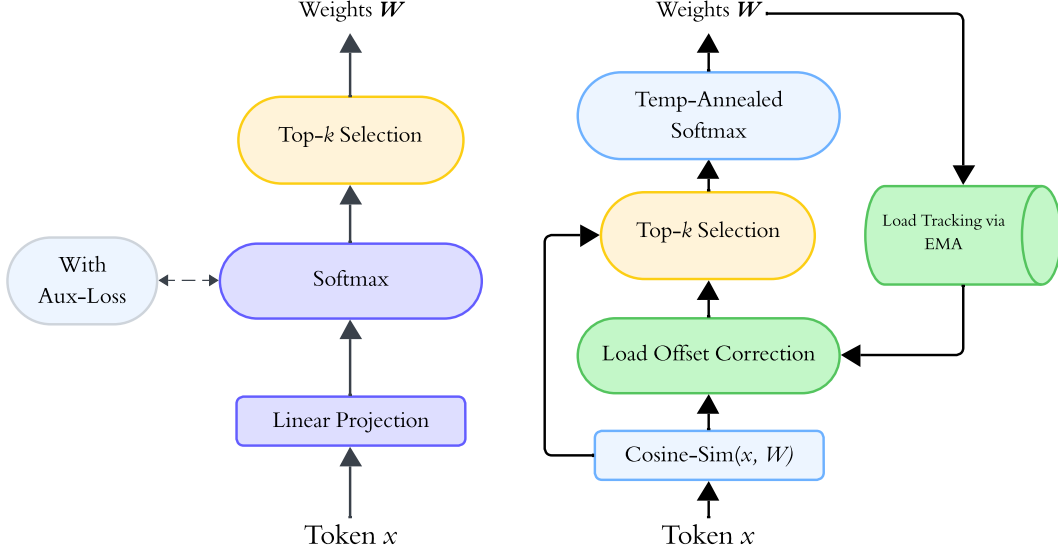


Figure 2: Router comparison. (a) Standard router: linear projection followed by softmax and top- k selection, trained with an auxiliary load-balancing loss. (b) SPAR router: cosine similarity scoring with EMA-based load offset and temperature-annealed softmax weighting, requiring no auxiliary loss.

with decay rate $\alpha \ll 1$ and initialization $L_i = k/N$. In distributed training, per-rank dispatch counts are summed across all workers via `all_reduce` before each EMA update so all replicas apply the same globally-pooled load signal.

Prototype Learning. The prototype matrix $W \in \mathbb{R}^{N \times D}$ is updated by gradient descent through the mixture weights w_i . Token activations and prototype rows are L2-normalized on the fly, so updates are constrained to changes in prototype direction rather than magnitude. Only experts in $\mathcal{S}(x)$ receive gradient from a given token, so prototypes specialize toward the directional centroid of their routed token population.

We initialize W from k-means centroids of a representative data batch, rather than random directions. Without this, identical expert outputs at initialization produce zero router gradient, and prototypes fail to differentiate. K-means initialization provides nonzero cosine differences from step 0, giving the router a meaningful signal before any expert has been trained.

4.2 Training Schedule

The temperature τ is annealed linearly from τ_0 to τ_{final} over the first portion of training, then held fixed. High temperature early on encourages exploration; lower temperature later sharpens weights as prototypes stabilize.

The penalty scale λ is calibrated once from data immediately after the learning-rate warmup completes. We compute σ_{cos} as the mean per-token inter-expert standard deviation of $\{\cos(x, W_i)\}_{i=1}^N$ and set

$$\lambda = \min(\sigma_{\text{cos}} \cdot N, \lambda_{\text{max}}), \quad (8)$$

where λ_{max} is a fixed cap. This ensures a per-expert load deviation of $1/N$ produces a logit correction of σ_{cos} , one standard deviation of the inter-expert cosine spread, making the penalty commensurate with the routing geometry regardless of model dimension. Calibrating post-warmup ensures prototypes have learned meaningful directions before σ_{cos} is measured; calibrating earlier underestimates it and yields λ permanently too weak. Both τ and λ are fixed after their respective scheduling events and require no further tuning.

5 Experiments, Key Metrics, and Results

5.1 Experimental Setup

All experiments use Qwen2-1.5B [16] as a frozen backbone, with MoE layers inserted at transformer layers 2, 6, 10, 14, 18, and 22, meanwhile other 22 layers are frozen (Figure 1). Each MoE layer contains 8 LoRA experts with rank $r = 32$ and scaling $\alpha = 32$, following the $\alpha = r$ convention recommended in LoRA [8], with top- $k = 2$ routing so each token is processed by exactly 2 experts per layer.

Dataset. All models are trained on FineWeb-Edu [17], a large-scale web-crawled corpus filtered for educational quality, tokenized using the Qwen2 tokenizer. Training sequences are formed by packing tokenized documents into fixed-length windows of 2048 tokens without padding; document boundaries are marked with an end-of-sequence token. FineWeb-Edu spans science, mathematics, humanities, and general web content, making it a demanding testbed for routing stability across heterogeneous input distributions.

Training Configuration. All runs share an identical training configuration. Models are optimized with AdamW [18] with a peak learning rate of 3×10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, and weight decay 0.1. The learning rate is warmed up linearly over 800 steps and decayed via cosine schedule over all the total training steps. Gradient norms are clipped to 1.0. Training uses a per-device batch size of 8 with gradient accumulation over 4 steps across 8 GPUs, giving an effective global batch size of 256 sequences ($\approx 524,288$ tokens) per step.

Baselines. We compare SPAR against three baselines, all sharing the same architecture, training configuration, and LoRA expert setup. The **standard** baseline uses top- k routing with a linear gate and auxiliary load-balancing loss ($\alpha = 0.01$) following Fedus et al. [2]. **Expert Choice** [10] inverts the routing direction: each expert selects its top tokens from the batch under a fixed capacity budget, guaranteeing load balance by construction. **DeepSeek V3** [7] uses sigmoid gating with per-expert bias terms updated each step to steer load toward uniformity without an auxiliary loss.

Infrastructure. We conduct experiments on GPU-based distributed training environments using NVIDIA accelerators. Our experiments were executed on $8 \times$ NVIDIA H100 GPUs. These configurations enable scaling from lightweight experiments to higher-capacity MoE training under a consistent setup.

We use two execution backends: Modal for rapid, containerized experimentation, and AWS Batch for larger-scale training jobs. Both environments use containerized CUDA runtimes, which ensures reproducibility and consistent dependency management across runs. Data preparation and training are decoupled into separate stages, enabling efficient resource utilization and reuse of preprocessed datasets across experiments.

Training is performed with DDP (Distributed Data Parallel) across GPUs. The implementation uses PyTorch distributed execution with optimized communication settings to reduce memory overhead and improve throughput. Mixed-precision training is enabled with bfloat16 on supported hardware, and TF32 is used to accelerate dense matrix operations. For Qwen2-based models, optimized attention kernels (FlashAttention [19]) further improve memory efficiency and runtime performance.

5.2 Evaluation Protocol

We evaluate SPAR using three complementary protocols: post-training perplexity evaluation, downstream task evaluation with the EleutherAI LM Evaluation Harness [20], and routing diagnostics.

Language Modeling. For offline language modeling evaluation, we report perplexity on Wikitext-103 [21] and the Pile validation set (pile-val) [22]. Perplexity is computed with a sliding-window scoring procedure over contiguous token streams using a maximum evaluation length of 2048 tokens and a stride of 512 tokens. The first window scores all valid positions, while subsequent windows score only newly introduced suffix tokens, ensuring that each token is counted exactly once while still benefiting from substantial left context.

Downstream Benchmarks. For downstream evaluation, we use the EleutherAI LM Evaluation Harness [20]. In the current setup, zero-shot evaluation is performed on HellaSwag [23], PIQA [24], WinoGrande [25], ARC-Easy and ARC-Challenge [26], BoolQ [27], and OpenBookQA [28], while five-shot evaluation is performed on MMLU [29]. We report the primary task metric provided by the harness, namely accuracy or normalized accuracy depending on the benchmark.

Routing Metrics. Beyond perplexity, we monitor three key metrics throughout training to assess load balancing and routing quality. We quantify expert utilization using **Effective Experts**, defined as $\exp(H)$ where H is the entropy of the expert usage distribution. This metric scales from 1 (selection collapse) to N (perfectly uniform load). Complementary to this, the **Gini coefficient** measures load imbalance, where 0 indicates equal utilization and 1 denotes a single-expert monopoly. Finally, we track the **LoRA delta norm** $\|\Delta_i\|_F = \|B_i A_i\|_F$ per expert, which measures how much each adapter has diverged from the frozen backbone. Divergence across experts indicates genuine specialization; uniform norms suggest experts remain interchangeable despite routing.

5.3 Main Results

SPAR Achieves the Best Perplexity Among Sparse Routers. We compare all router variants under identical conditions: Qwen2-1.5B backbone, LoRA-MoE with $N=8$ experts, top- $k=2$, rank $r=32$, 19k training steps on FineWeb-Edu. Table 1 shows that SPAR achieves the lowest WikiText-103 perplexity (10.99), outperforming Top-K (11.88) by 0.89 and DeepSeek (12.20) by 1.21, with no auxiliary loss. On Pile, SPAR ties the best baseline (7.41 vs. 7.43 for DeepSeek). Across eight downstream tasks, SPAR leads on five and ties on two, yielding an average of 0.5957 against DeepSeek’s 0.5961, a gap of less than 0.1%. Expert Choice underperforms on all metrics; fixed expert capacity causes token-dropping and training instability, reflected in delta norms reaching the thousands.

Table 1: Main results for router variants on Qwen2-1.5B LoRA-MoE (FineWeb-Edu, 19k steps). Higher is better ($\uparrow$) and lower is better ($\downarrow$). **Bold** indicates best; underline indicates tie.

Metric	Dense (frozen)	Top-K	DeepSeek	Expert Choice	SPAR (ours)
Trainable Params (M) $\downarrow$	0	48.44	48.44	48.44	48.44
Train Tokens (B)	—	9.96	9.96	9.96	9.96
PPL (WikiText-103) $\downarrow$	9.67	11.88	12.20	14.78	10.99
PPL (Pile) $\downarrow$	7.32	7.44	7.43	11.87	7.41
HellaSwag (0-shot) $\uparrow$	—	0.6488	0.6520	0.2582	0.6523
PIQA (0-shot) $\uparrow$	—	<u>0.7633</u>	0.7595	0.5060	0.7633
WinoGrande (0-shot) $\uparrow$	—	0.6527	<u>0.6535</u>	0.5233	0.6535
ARC-Easy (0-shot) $\uparrow$	—	0.6705	0.6793	0.2677	0.6843
ARC-Challenge (0-shot) $\uparrow$	—	0.3925	0.3942	0.2415	0.3823
BoolQ (0-shot) $\uparrow$	—	0.7095	0.7089	0.3838	0.7009
OpenBookQA (0-shot) $\uparrow$	—	0.3680	<u>0.3720</u>	0.2900	0.3720
MMLU (5-shot) $\uparrow$	—	0.5505	0.5567	0.3412	0.5491
Avg. (8-task) $\uparrow$	—	0.5945	0.5961	0.3515	0.5957

SPAR Trades Uniform Load for Expert Specialization. Figure 3 (left) shows that Top-K, DeepSeek, and Expert Choice maintain near-perfect balance ($\text{eff_E_hard} \approx 8.0$), while SPAR allows moderate non-uniformity (7.2-7.9 across layers, most pronounced at L18: 7.34 and L22: 7.18). This is not a failure of SPAR’s load mechanism. When all experts receive equal token mass, their prototype vectors are forced to Voronoi cell boundaries where pairwise cosine differences approach zero, structurally bounding routing confidence regardless of temperature. Enforcing $\text{eff_E_hard} = 8.0$ therefore trades routing expressivity for load accounting. The right panel shows the consequence: SPAR’s cross-expert variance in $\|\Delta W\|_F$ is 2-4 $\times$ higher than Top-K at every layer and grows with depth, indicating substantially stronger expert specialization.

Gradient Coherence Links Token Frequency to Weight Drift. Figure 4 (left) isolates the mechanism at layer 18, where specialization is most pronounced. A clear inverse relationship emerges

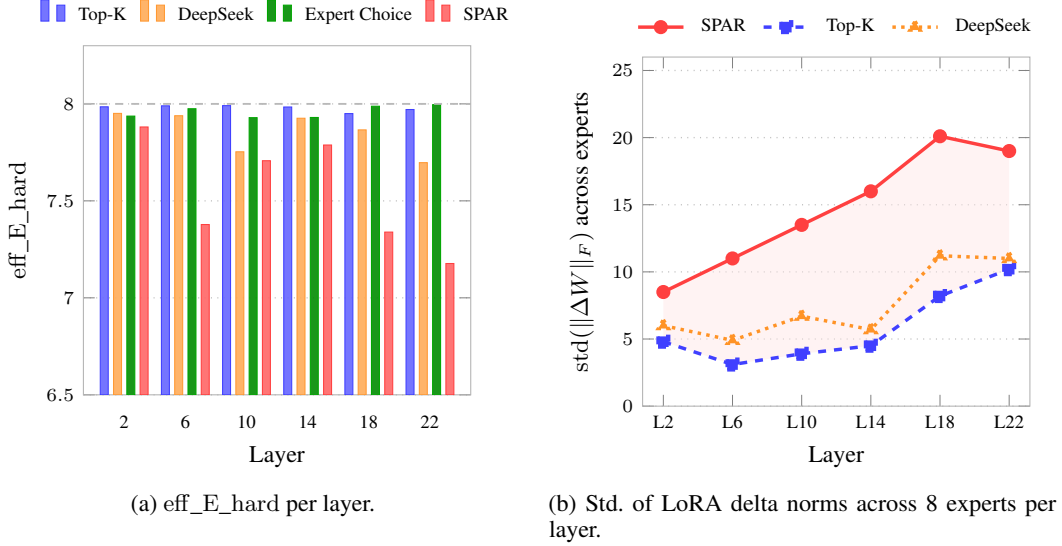


Figure 3: **Left:** Top-K, DeepSeek, and Expert Choice enforce near-uniform load; SPAR’s moderate non-uniformity in deeper layers reflects cosine routing geometry, not a balance failure. **Right:** SPAR’s reduced load uniformity directly enables stronger expert differentiation.

between routing frequency and weight drift: low-load experts (E3, E5, E6; $<8\%$ of tokens) drift far from the frozen base MLP ($\|\Delta W\|_F > 73$), while high-load experts (E0, E2; $>15\%$) remain close. A low-load expert serves a narrow, consistent token distribution, so its parameter updates accumulate in a stable direction and produce large net drift. A high-load expert processes the full token diversity; opposing gradient directions partially cancel and suppress drift. SPAR does not enforce specialization directly; it emerges as a byproduct of the load distribution the penalty mechanism induces.

Forced Uniformity Suppresses This Mechanism in Competing Routers. Figure 4 (right) shows why Top-K cannot exhibit gradient coherence. Its auxiliary loss holds routing entropy at ≈ 1.0 across all layers, forcing every expert to process a maximally diverse token distribution. SPAR’s entropy falls naturally in deeper layers (L18: 0.933, L22: 0.920), reflecting self-organized expert roles that the representation geometry supports. DeepSeek applies a similar uniform pressure through a discrete sign-update bias that shifts all routing logits at every step, regardless of whether balance has been achieved; SPAR’s penalty vanishes at the load equilibrium and applies pressure only when needed. The perplexity advantage SPAR holds over Top-K (10.99 vs. 11.88) is the downstream consequence of permitting gradient coherence to develop where the frozen backbone’s geometry permits it.

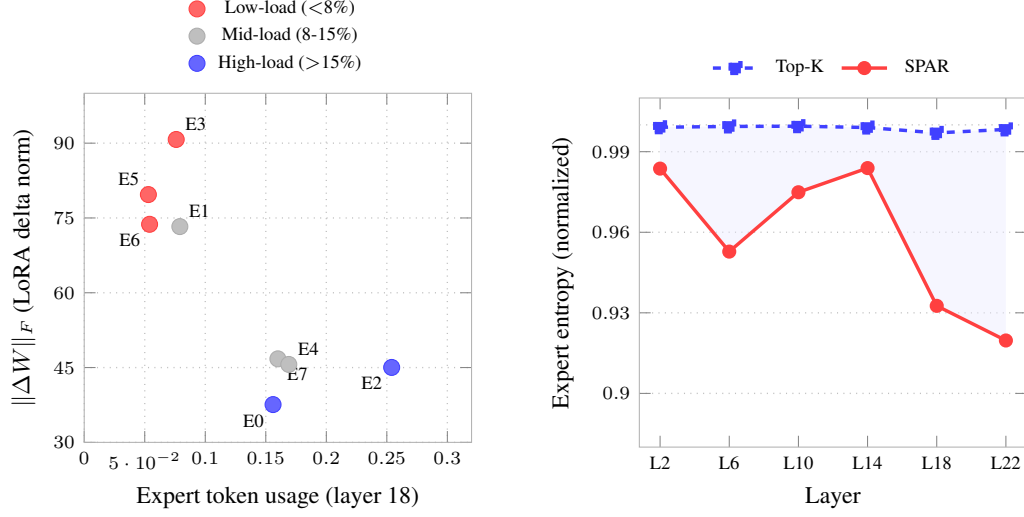
5.4 Ablation Studies

Penalty Formulation. The load correction in Equation 4 can take two forms: a one-sided penalty $\max(0, L_i - k/N)$ that only suppresses overloaded experts, or the symmetric form $(L_i - k/N)$ that equally boosts underloaded ones. We ablate this choice while holding all other hyperparameters fixed. Table 2 reports 6-layer mean routing metrics at step 19k.

Table 2: Effect of penalty formulation on routing quality. Metrics are 6-layer means at step 19k on Qwen2-1.5B / FineWeb-Edu. Task performance is reported in Table 1.

Penalty	Fair share	eff_E_hard $\uparrow$	Gini $\downarrow$	Entropy $\uparrow$	Train PPL $\downarrow$
One-sided $\max(0, L_i - k/N)$	$1/N$	5.74	0.384	0.875	11.32
Symmetric $(L_i - k/N)$ (ours)	k/N	7.55	0.215	0.958	11.36

The symmetric formulation raises mean effective expert utilization from 5.74 to 7.55 (+31%) and reduces the Gini coefficient from 0.384 to 0.215 (−44%), while training perplexity changes by less than 0.04. The asymmetric form cannot redistribute load bidirectionally: underloaded experts receive



(a) Token usage vs. $\|\Delta W\|_F$ for SPAR experts at layer 18.

(b) Normalized routing entropy per layer.

Figure 4: **Left:** The inverse correlation confirms gradient coherence as the specialization mechanism. **Right:** Top-K’s auxiliary loss pins entropy to ≈ 1.0 ; SPAR’s self-organized entropy drop in deeper layers is the structural precondition for expert specialization.

no competitive advantage, so the routing equilibrium settles at lower utilization. The improvement is most pronounced in deeper layers, where imbalance was worst (L18: 5.11 $\rightarrow$ 7.34; L22: 5.66 $\rightarrow$ 7.18).

Expert Stress Regularization. We evaluated expert stress, defined as the coefficient of variation of per-token cosine distances within each expert, as an additional routing-logit penalty with weight μ_{stress} . At $\mu_{\text{stress}} = 0.5$, eff_E_hard dropped from 7.5 to 5.8 within 2000 steps and the Gini coefficient rose from 0.11 to 0.41. This happens because high-traffic experts collect many more observations, which biases online variance estimates upward relative to low-traffic experts and causes excessive suppression. At $\mu_{\text{stress}} = 0.1$, we observed no measurable effect on routing. We therefore set $\mu_{\text{stress}} = 0$ and it remains a monitoring metric, computed with Welford’s online statistics [30].

6 Conclusion

We presented SPAR, a load-balancing router for sparse LoRA-MoE adaptation that embeds load correction directly into the routing scores rather than into the training objective. By penalizing overloaded experts and boosting underloaded ones symmetrically, SPAR achieves balance without an auxiliary loss. The penalty vanishes at equilibrium, leaving routing governed purely by cosine similarity once balance is achieved. On Qwen2-1.5B with eight LoRA experts, SPAR achieves the lowest WikiText-103 perplexity among all sparse router variants and matches the best baseline on Pile, while inducing 2-4 $\times$ greater expert specialization than load-constrained routers. The single free parameter, λ , is calibrated automatically from data and requires no manual tuning.

7 Future Work and Next Steps

The main open question is how SPAR scales beyond Qwen2-1.5B. In particular, we need to verify whether cosine routing geometry and automatic λ calibration remain stable at 7B–8B scale, where hidden dimensions are larger and the inter-expert cosine spread σ_{cos} shrinks as $O(1/\sqrt{D})$. A direct next step is validation on Llama-3 8B [31] (or similar models), including tests of a λ_{min} floor. Additional ablations should measure sensitivity to EMA decay α , number of MoE layers, and the need for k-means prototype initialization at larger N .

Team Reflection

This project required our team to take ownership of the full research lifecycle, including problem formulation, infrastructure setup, experiment design, and the clear communication of results. Managing this end-to-end process emphasized the importance of coordination and disciplined execution in an industrial research setting.

Our project originated from a vision that proved to be a little too ambitious for our current scope: building a **Thermodynamic Mixture-of-Experts (T-MoE)**. This system was designed to eliminate the central router entirely, aiming to prove that simple biological rules, such as experts getting “tired” (fatigue) or splitting when overwhelmed (fission), could naturally solve problems like catastrophic forgetting and load balancing. However, as we moved into implementation, we realized that a model that “grows” its own topology was too broad to tackle within the given constraints. This forced us to reflect on the practicalities of research and pivot toward a specific, high-impact part of that vision: using a fatigue-inspired mechanism to solve routing collapse.

This refined focus led to **SPAR**. By applying a logit-level penalty based on real-time load tracking, we emulated expert fatigue and achieved good results without the complexity of a full biological simulation. We maintained rigorous experiment logs to support reproducibility and faster iteration. This systematic approach helped us identify failure patterns, such as the speed mismatch in routing dynamics, and refine our direction based on evidence. Translating high-level research papers into working implementations also exposed the gap between theoretical descriptions and practical details, particularly in training stability and routing behavior.

Our technical foundation was built directly on course materials. Early assignments like **Karpathy’s NanoGPT** were essential for helping us structure our training pipelines. The course coverage of **Mixture-of-Experts (MoE)**, including systems like **GShard** and **DeepSeek**, directly informed our architectural decisions. Furthermore, the **LoRA tutorial** provided the critical mathematical understanding of parameter-efficient fine-tuning needed for our implementation.

As we scaled our experiments, we directly applied course material on **Distributed Training**. **Cameron R. Wolfe’s guest lecture** was particularly impactful, as his explanations of data parallelism, checkpointing strategies, and the logistics of training large-scale models in a real-world setup were instrumental as we moved to multi-GPU environments. Implementing **Distributed Data Parallel (DDP)** helped us understand parallel training and system-level considerations, such as synchronizing EMA load tracking across 8 H100 GPUs. We also used **AI-based code assistance** to accelerate iteration, allowing us to focus more on experimental design.

In hindsight, there are several things we would change. While we eventually identified the scaling behavior of the inter-expert cosine spread $\sigma_{\cos}$ as $O(1/\sqrt{D})$, we spent significant time debugging the penalty magnitude before realizing how hidden dimension size affects routing geometry. If we were to restart, we would prioritize a more modular “plugin” architecture for our routing strategies from day one, as our initial monolithic scripts made it difficult to run clean ablations on the EMA decay rate α and prototype initialization. We would also incorporate more aggressive checkpointing and validation earlier in the pipeline to catch the speed-mismatch in routing dynamics before they led to total collapse.

Beyond the technical implementation, this project served as a case study in research project management under strict deadlines. We faced a constant tension between the desire to explore the broader T-MoE theoretical framework and the practical necessity of producing a stable, baseline-validated solution like SPAR. Managing this required a disciplined approach to time-boxing our exploratory phases and setting hard-stops for architectural changes. We learned that in a research pipeline, the bottleneck is often not the GPU compute time, but the human time required to analyze logs and adjust hypotheses. By establishing a clear internal roadmap and prioritizing the most well-motivated experiments, we were able to transition from a conceptual model to a high-performance routing implementation without succumbing to feature creep. This experience highlighted that effective research requires as much project management and risk mitigation as it does mathematical derivation.

Ultimately, seeing SPAR achieve significantly better balance than standard methods confirmed that narrowing our scope from the broad T-MoE concept to a specific, well-motivated solution was the right decision for the project’s success.

References

- [1] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- [2] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [3] Barret Zoph, William Fedus, Yuhuai Zhou, and Noam Shazeer. Designing effective sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.
- [4] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch Roux, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [5] Dmitry Lepikhin, Hyukjoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, and Noam Shazeer. Gshard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
- [6] Zihan Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024.
- [7] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [9] Deng Li et al. Mixlora: Enhancing large language models fine-tuning with lora based mixture of experts. *arXiv preprint arXiv:2404.15159*, 2024.
- [10] Yanqi Zhou, Tao Lei, Han Liu, Nan Huang, Vincent Zhao, Andrew Dai, et al. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [11] Damai Dai et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- [12] X.Y. Han and Yuan Zhong. A primal–dual perspective on auxiliary-loss-free load balancing in mixture-of-experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [13] Wei Yang et al. Lpr: Variational prototype routing for post-hoc mixture-of-experts load balancing. *arXiv preprint*, 2025.
- [14] Nouredine Zadouri et al. Molara: Mixture of low-rank adapters for parameter-efficient fine-tuning. *arXiv preprint arXiv:2402.12851*, 2024.
- [15] Shihan Dou et al. Loramoe: Alleviating world knowledge forgetting in large language models via moe-style task specialization. *arXiv preprint arXiv:2402.18174*, 2024.
- [16] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Dayiheng Pan, Haoran Li, Fei Huang, Huajian Gao, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [17] Guilherme Penedo, Dmytro Kydluk, Loubna Ben Allal Mishra, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.
- [18] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [19] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.
- [20] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sidney Black, Anthony DiPofi, Charles Foster, Andrew Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aidan Skowron, Lin Sutawika, Eric Tang, Ananya Thite, Ben Wang, Kevin Wang, and Andy Zou. lm-evaluation-harness: A framework for few-shot evaluation of language models. *GitHub repository*, 2024. URL <https://github.com/EleutherAI/lm-evaluation-harness>.
- [21] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.

- [22] Leo Gao, Stella Biderman, Sidney Black, Andrew Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Ananya Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- [23] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4791–4800, 2019.
- [24] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 34, pages 7432–7439, 2020.
- [25] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 35, pages 8732–8740, 2021.
- [26] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [27] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.
- [28] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2381–2391, 2018.
- [29] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- [30] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4 (3):419–420, 1962.
- [31] AI at Meta. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Appendix A: Hyperparameters

Table 3: Hyperparameters shared across all runs.

Hyperparameter	Value	Notes
<i>Model</i>		
Backbone	Qwen2-1.5B	Frozen during training
MoE layer indices	2, 6, 10, 14, 18, 22	6 of 28 transformer layers
Number of experts	8	Per MoE layer
Top- k	2	Experts activated per token
LoRA rank	32	
LoRA α	32	Effective scale = $\alpha/r = 1.0$
LoRA dropout	0.0	
<i>SPAR-specific</i>		
Initial temperature τ_0	0.5	
Final temperature τ_f	0.10	
Temperature anneal steps	14,000	
EMA coefficient α	0.01	≈ 100 -step smoothing window
λ calibration step	1,000	warmup + 200
Initial λ	0.1	Pre-calibration value
Exploration noise σ	0.0	Omitted: $\sigma_{\cos} < \sigma$ at $D = 1536$
Prototype init	k-means	From layer activations before training
<i>Standard baseline-specific</i>		
Aux loss coefficient	0.01	Switch Transformer default
Gate temperature	1.0	
<i>DeepSeek baseline-specific</i>		
Bias update rate γ	1×10^{-3}	DeepSeek V3 paper value
Gating function	Sigmoid	DeepSeek V3 formulation
<i>Expert Choice baseline-specific</i>		
Capacity factor	2	Each expert selects $2 \times (T/N)$ tokens
Gate temperature	1.0	

Appendix B: Theoretical Analysis of SPAR

We provide formal derivations for the key properties described in Section 4.

Token Cluster Geometry and Initialization. In a frozen LLM backbone, hidden states at a given MoE layer concentrate around high-density regions corresponding to linguistic concepts, rather than uniformly on the hypersphere. Initialization via K-means++ sets expert prototypes $\hat{W}_i$ to Fréchet means of these clusters, ensuring nonzero inter-expert cosine variance $\sigma_{\cos}^2 > 0$ from initialization and avoiding random deadlock. Here, $\hat{x} = x/\|x\|$ and $\hat{W}_i = W_i/\|W_i\|$.

Proposition 1. *Under the SPAR output weight formulation (Equation 5), gradient descent on W_i induces a soft spherical k-means update with assignment weights w_i .*

Proof. The MoE output is $y = \sum_{i \in S(x)} w_i f_i(x)$ where

$$w_i = \frac{\exp(\cos(x, W_i)/\tau)}{Z}.$$

Let $z_i = \cos(x, W_i)/\tau$. The softmax Jacobian is

$$\frac{\partial w_i}{\partial z_j} = w_i(\delta_{ij} - w_j).$$

Focusing on the dominant diagonal term for clarity, we obtain

$$\frac{\partial w_i}{\partial z_i} = \frac{w_i(1 - w_i)}{\tau}.$$

The gradient of cosine similarity is

$$\frac{\partial \cos(x, W_i)}{\partial W_i} = \frac{\hat{x} - \cos(x, W_i) \hat{W}_i}{\|W_i\|}.$$

Combining terms gives

$$\frac{\partial \mathcal{L}}{\partial W_i} \propto \sum_{x: i \in \mathcal{S}(x)} \alpha_i(x) \cdot (\hat{x} - \cos(x, W_i) \hat{W}_i),$$

where $\alpha_i(x) > 0$ aggregates scalar factors. This matches the soft-assignment update of spherical k -means, whose fixed point is the weighted Fréchet mean on S^{D-1} . $\square$

Corollary 1. *No additional auxiliary loss or explicit centroid supervision is required. The language modeling objective alone drives W_i toward the optimal representative direction of its routed token population.*

Proposition 2. *Under the calibration $\lambda = \sigma_{\cos} \cdot N$ (Equation 8), SPAR can correct maximum load imbalance regardless of a token’s cosine preference.*

Proof. Let expert i^* be maximally overloaded ($L_{i^*} \rightarrow 1$) and expert j underloaded ($L_j \rightarrow 0$). The load-corrected logits are

$$z_{i^*}(x) = \cos(x, W_{i^*}) - \lambda(1 - k/N), \quad z_j(x) = \cos(x, W_j) + \lambda \cdot k/N.$$

Expert j is preferred when $z_j > z_{i^*}$, which requires

$$\cos(x, W_{i^*}) - \cos(x, W_j) < \lambda.$$

Thus, any preference gap smaller than λ can be overridden by load balancing. With $N=8$ and $k=2$, the effective correction is

$$\Delta z = \sigma_{\cos}(N - k) = 6 \sigma_{\cos},$$

which exceeds the typical $1-2 \sigma_{\cos}$ gap between a token’s top experts. $\square$

Appendix C: Empirical Analysis of Expert Specialization

To evaluate whether SPAR induces meaningful expert specialization, we analyze token distributions under SPAR v3 on Wikitext-103 [21]. Per-expert specialization is defined as $\text{Spec} = 1 - \text{TTR}$ (type-token ratio), where higher values indicate a more homogeneous token distribution.

Table 4: Per-layer token distribution statistics under SPAR v3 (Wikitext-103). Gini measures load imbalance; Mean Spec. denotes average specialization across experts.

Layer	Mean Tokens	Gini	Mean Spec.	Most Specialized
2	8418	0.108	0.793	E6 (0.959)
6	8418	0.358	0.761	E3 (0.958)
10	8418	0.337	0.758	E6 (0.948)
14	8418	0.142	0.789	E7 (0.911)
18	8418	0.312	0.764	E1 (0.931)
22	8418	0.366	0.777	E5 (0.925)

Three patterns emerge. First, *functional specialization*: experts consistently capture distinct token classes across layers. Some specialize in punctuation and whitespace ($\text{Spec} > 0.94$), others in numerics, and others in domain-specific vocabulary (e.g., sports terms or named entities). Second, *load imbalance varies by depth*: layer 6 shows the highest imbalance ($\text{Gini} = 0.358$), while earlier layers (2, 14) are more balanced ($\text{Gini} \approx 0.1$). Third, *semantic diversity increases with depth*: mean specialization decreases from layer 2 (0.793) to layer 10 (0.758), consistent with higher-level abstraction in deeper layers. Notably, punctuation-focused experts persist across all layers, acting as stable structural token sinks.